

Lanka Ride — what each member built

Group 2026-Y2-S1-MLB-B13G2-05 · SE2030 · Spring Boot 3.3, JSP, Spring Security, MySQL `lanka_ride`

Lanka Ride is a vehicle rental web application. A customer searches a branch, asks to book a car, a supervisor approves or denies the request, finance takes payment, and staff record pickup and return. Fleet staff can take a car offline for maintenance. Support staff answer tickets. Managers read reports.

Every feature uses the same path. The browser shows a JSP page. The form posts to a controller. The controller calls a service. The service checks the rules and asks a repository to read or write MySQL. The controller then redirects, and the next JSP draws the result. A hidden CSRF token sits on every POST form. This guide does not change any of that Java.

Who owns what

Member	ID	Role	Use case	Package
Sakalasooriya S.A.A.A.	IT25102266	Developer	UC-02 Vehicle management only: catalogue, create, update, transfer, retire, and delete.	<code>vehicle</code>
De Silva D.L.K.C.	IT25102264	Developer	UC-01 Fleet maintenance	<code>fleet</code>
Samaranayake P.I.S.	IT24101349	Scrum Master	UC-03 Booking and reservation	<code>booking</code>
Kavindi P.D.N.	IT25102265	Developer	UC-04 Payment and billing	<code>payment</code>
Wickramasinghe R.D.W.K.G.S.	IT24101008	Product Owner	UC-05 Dashboard and reports	<code>dashboard</code>
Pahasara Y.A.P.	IT24102871	Developer	UC-06 Support, notifications, password reset, role menus, and adding branches and staff.	<code>support</code>

How a page is built

`header.jsp` and `footer.jsp` wrap every screen. The header chooses a marketing bar on the home page, a short Lanka Pay bar on the card page, or the signed-in menu everywhere else. `sec:authorize` hides links the current role must not see. `app.css` is the only stylesheet. Photos sit behind the home page, login, register, the customer dashboard, and the maintenance and admin screens. The forms and numbers stay in white cards.

`HomeController` at `/` loads the branches into `home/index.jsp`. The search form is a GET to `/vehicles`. That is Sakalasooriya's catalogue, reached from the shared home page.

Demo accounts

Username	Password	Lands on
customer	customer123	/app My trips
admin	admin123	/dashboard
fleet	fleet123	/dashboard and /maintenance
supervisor	super123	/dashboard and bookings
finance	finance123	/dashboard , payments, reports
operations	ops123	/dashboard , reports, fleet defaults

Public registration always creates a CUSTOMER. Staff accounts are created only on the admin screen.

1. Sakalasooriya S.A.A.A. — vehicle management only

IT25102266 owns vehicle management and nothing else in this guide. That is the car catalogue: search, the detail page, add, edit, transfer between branches, retire, and permanent delete. A visitor can search cars without signing in. Creating, editing, transferring, retiring, or deleting a car needs ADMIN, FLEET_COORDINATOR, or OPERATIONS_MANAGER. Branches, staff accounts, login, and support tickets belong to other members.

CRUD

Action	URL	What it does
Read	GET <code>/vehicles</code>	Search by branch, category, dates, and “available only”. Retired cars stay hidden.
Read	GET <code>/vehicles/{id}</code>	One car: price, seats, photo, branch, status, transfer history.
Create	GET/POST <code>/vehicles/new</code> and POST <code>/vehicles</code>	Save a new car on a branch. Registration number must be unique.
Update	GET/POST <code>/vehicles/{id}/edit</code> and POST <code>/vehicles/{id}</code>	Change the same car. The plate check ignores its own row.
Update	POST <code>/vehicles/{id}/transfer</code>	Move the car to another branch and write a <code>branch_transfers</code> row.
Update	POST <code>/vehicles/{id}/retire</code>	Status becomes RETIRED. The row stays in the database.
Delete	POST <code>/vehicles/{id}/delete</code>	Permanent delete. Payments, bookings, maintenance, and transfers for that car are removed first.

Screens

- `vehicle/list.jsp` — filter form and a card for each car. Search is a GET, so the address bar holds the filters.
- `vehicle/detail.jsp` — photo, specs, and the staff buttons for edit, transfer, retire, and delete.
- `vehicle/form.jsp` — Spring `form:form` bound to a `Vehicle`. The same page creates and edits. A small script fills brand and model from the saved fleet defaults. Those defaults are Wickramasinghe’s catalogue, not this module.

Java, in plain words

- `Vehicle` is the table `vehicles`. Status is AVAILABLE, UNAVAILABLE, MAINTENANCE, or RETIRED. A car belongs to one `Branch`.
- `VehicleRepository.search` is a JPQL query. Blank filters are ignored. It never returns RETIRED cars.
- `VehicleService.search` runs that query, then drops cars whose bookings overlap the chosen dates. Pending, approved, and ongoing bookings block the car.
- `VehicleService.create` and `update` clean text, check the plate, check seats from 1 to 20, and check money through `InputChecks`.

- `VehicleService.transfer` refuses a retired car and a move to the same branch. It saves a `BranchTransfer` with who did it and when.
- `VehicleController` is thin. It reads the form, calls the service, and redirects with a flash message. That is the post-redirect-get pattern, so a refresh does not save the car twice.
- His Java files are `Vehicle`, `VehicleController`, `VehicleService`, `VehicleRepository`, `Branch`, `BranchRepository`, `BranchTransfer`, and `BranchTransferRepository`. `Branch` is the depot a car belongs to. Adding a new depot is not this module.

2. De Silva D.L.K.C. — fleet maintenance

He owns service records. Opening a job sets the car to MAINTENANCE so a new booking is refused. Closing the job sets the car back to AVAILABLE. Only ADMIN and FLEET_COORDINATOR can open /maintenance.

CRUD

Action	URL	What it does
Read	GET /maintenance	List every service record.
Read	GET /maintenance/{id}	One record, the car, and the close form.
Create	GET /maintenance/new then POST /maintenance	Open a job. If the car has a future booking, the form asks the user to confirm.
Update	GET /maintenance/{id}/edit then POST /maintenance/{id}	Edit an open job. A closed job cannot be edited.
Update	POST /maintenance/{id}/close	Store the final cost, mark CLOSED, and make the car AVAILABLE again.
Delete	POST /maintenance/{id}/delete	Remove that record.
Update	POST /maintenance/reminders	Send due-service reminders into the notification log.

Screens

- fleet/list.jsp — cards for each job, plus “Add record” and “Send due reminders”.
- fleet/form.jsp — vehicle, service type, dates, cost, description, and mechanics. The same form edits an existing job.
- fleet/detail.jsp — the job, a close button, and delete. Close posts the final cost.

These three pages use the garage photo behind the white cards.

Java, in plain words

- MaintenanceRecord stores the service type, dates, costs, description, mechanics, status (OPEN, IN_PROGRESS, CLOSED), and the vehicle.
- MaintenanceService.create looks for bookings that still need the car. Without the confirm flag it stops and tells the controller to show the warning. With the flag it saves the job and sets the vehicle status to MAINTENANCE.
- MaintenanceService.close writes the final cost, sets CLOSED, and sets the vehicle back to AVAILABLE.
- MaintenanceService.openInspectionFromReturn is called from the payment return path when staff record a mileage or damage discrepancy. The booking stays. A new maintenance row is opened.
- MaintenanceReminderScheduler can call the same reminder method on a timer. The button on the list page calls it immediately.

- `MaintenanceController` only gathers form fields, calls the service, and redirects. The conflict warning is an exception turned into a message on the form.

3. Samaranayake P.I.S. — bookings

He is Scrum Master and he owns the reservation. A booking is a request, not an instant rental. It starts PENDING and UNPAID. A supervisor approves or denies it. The customer or staff can cancel while it is still pending or approved. The monitor page is read-only.

CRUD and the status changes

Action	URL	What it does
Read	GET /bookings	A customer sees only their trips. Staff see every booking.
Read	GET /bookings/pending	Staff queue of PENDING requests.
Read	GET /bookings/monitor	Operations view. No buttons that change data.
Read	GET /bookings/{id}	One booking. Staff see approve and deny. The customer sees pay when it is approved.
Create	GET /bookings/new then POST /bookings	Customer asks for a car, a branch, and two dates. Status becomes PENDING.
Update	POST /bookings/{id}/approve	PENDING becomes APPROVED. Payment status becomes PENDING_PAYMENT.
Update	POST /bookings/{id}/deny	PENDING becomes DENIED, with a reason.
Update	POST /bookings/{id}/cancel	Customer or staff cancel a pending or approved trip.
Delete	POST /bookings/{id}/delete	Permanent delete of that booking when the rules allow it.

Pickup and return mileage are stored on the booking, but the buttons live on Kavindi's invoice page. The two modules share the `Booking` row.

Screens

- `booking/list.jsp` — cards, status pills, and filters for pending or all.
- `booking/form.jsp` — dates and the chosen car. The page script stops a return date that is before pickup.
- `booking/detail.jsp` — the decision screen. Approve and deny are separate POST forms. The pay link goes to the invoice.
- `booking/monitor.jsp` — the same trips in a staff board, without edit buttons.

Java, in plain words

- `Booking` holds the customer, vehicle, branch, dates, status, payment status, mileage, fuel, late fee, damage charge, and a discrepancy note.
- `BookingService.create` rejects a car that is in maintenance, unavailable, or retired. It then calls `countOverlapping`. If another pending, approved, or ongoing booking covers those dates, the save is refused.
- `approve` only works on PENDING, and it checks the car status again. A car that went into the workshop after the request cannot be approved.

- `deny` stores the staff reason. `cancel` checks that the caller is the customer or a staff member.
- `BookingRepository.countOverlapping` is the date rule: pickup is on or before the end, and return is on or after the start, and the status is in the blocking list.
- `BookingController` reads the signed-in name and passes it to the service. Customers and staff share the same URLs. The service and `@PreAuthorize` decide who may approve.

4. Kavindi P.D.N. — payments

She owns the money. Each movement is its own row: rental, deposit, late fee, damage, or refund. The invoice reads those rows. The card number is checked by a sandbox gateway and then thrown away. Only the brand and the last four digits are stored.

CRUD

Action	URL	What it does
Read	GET <code>/payments</code>	Staff ledger of every transaction.
Read	GET <code>/payments/booking/{id}</code>	The invoice for one booking.
Create	GET/POST <code>/payments/booking/{id}/checkout</code>	Lanka Pay. Visa, Mastercard, or American Express. A test card ending 4242 is approved. A card ending 0002 is declined.
Read	GET <code>.../checkout/success</code>	Receipt after a successful charge.
Update	POST <code>.../pickup</code>	Staff record pickup mileage and fuel. Booking becomes ONGOING.
Update	POST <code>.../return</code>	Staff record return mileage, fuel, late fee, and damage. Booking becomes COMPLETED. Extra charges become new payment rows.
Create	POST <code>.../refund</code>	Finance or admin refund an allowed booking. A refund row is inserted. The old payment row is not edited.

Screens

- `payment/list.jsp` — the ledger cards: type, amount, who recorded it, and success or failure.
- `payment/invoice.jsp` — amount due, pay button, pickup form, return form, and refund form. Which block shows depends on the booking status.
- `payment/gateway.jsp` — the card form. It uses the short Lanka Pay header.
- `payment/gateway-success.jsp` — the approved receipt and the link back to the invoice.

Java, in plain words

- `PaymentTransaction` is one line of the ledger: type, amount, gateway reference, brand, last four digits, success flag, and the staff or customer who caused it.
- `DummyGateway.charge` checks length, the Luhn digit, the brand, the name, the expiry, and the CVV. It returns a decision. It does not save the full card number.
- `PaymentService.payApprovedBooking` only runs when the booking is APPROVED and waiting for payment. A decline does not mark the booking paid.
- `recordPickup` stores the mileage and fuel and moves the booking to ONGOING.
- `completeReturn` stores return readings, optional late fee and damage, inserts those charges, and marks the booking COMPLETED. A discrepancy note can open a maintenance job through De Silva's service.

- `refund` inserts a refund row and sets the payment status to REFUNDED when the booking is allowed to be refunded.
- `PaymentController` passes the signed-in username into the service so the ledger can show who took the payment.

5. Wickramasinghe R.D.W.K.G.S. — dashboard and reports

He is Product Owner and he owns the numbers. The staff dashboard counts cars and pending bookings from the live tables. Reports read payments and vehicles. They do not keep a second copy of the money. Operations can also maintain the fleet defaults that the vehicle form uses.

CRUD

Action	URL	What it does
Read	GET /dashboard	Staff home: total cars, available, in maintenance, branches, pending bookings, and links for that role.
Read	GET /app	Customer home: active trips, trips waiting for payment, and the latest bookings. CUSTOMER only.
Read	GET /reports	Daily, monthly, or annual figures, optional branch filter, utilization, and location rows.
Read	GET /reports/export	The same report as a CSV file.
Read	GET /dashboard/catalog	List categories, models, and service types.
Create	POST /dashboard/catalog	Add one default.
Update	POST /dashboard/catalog/{id}	Edit one default.
Delete	POST /dashboard/catalog/{id}/delete	Remove one default. ADMIN and OPERATIONS_MANAGER only.

Screens

- `dashboard/index.jsp` — the operations dashboard. Metric cards on top, then a workspace of links. The garage photo sits behind the top of the page.
- `dashboard/customer.jsp` — “My trips”. Counts and recent booking cards over the fleet photo.
- `dashboard/reports.jsp` — period buttons, branch filter, totals, and a print-friendly table.
- `dashboard/catalog.jsp` — three kinds of defaults, with add, edit, and delete.

Java, in plain words

- `DashboardController.dashboard` calls `count()` and `countByStatus` on the vehicle repository and `countPending` on the booking service. It puts those numbers on the model. The JSP only prints them.
- `customerHome` loads that customer’s bookings and counts how many are active and how many are approved but unpaid.
- `ReportService.buildReport` chooses a date window from “daily”, “monthly”, or “annual”, then sums successful payment rows and counts bookings and vehicles. A branch id narrows the query.
- `ReportService.toCsv` turns that same map into text. `ReportsController` returns it with content type `text/csv`.
- `FleetDefault` is a small lookup row: kind (category, model, or service type) and a label. `FleetDefaultService` lists, saves, and deletes them. The vehicle form reads the lists so staff do not invent a new spelling of the same model every time.

6. Pahasara Y.A.P. — support and notifications

He owns tickets, the notification log, password reset, the role menus, and the admin screen that adds a branch or a staff user. Email is not sent to a real inbox. Each attempt is a row in the notification log, and a failed row can be retried. Public registration always creates a customer. Only this admin screen can create a staff account.

CRUD

Action	URL	What it does
Read	GET <code>/support</code>	A customer sees their tickets. Staff see the queue.
Create	GET <code>/support/new</code> then POST <code>/support</code>	Open a ticket. Status starts at OPEN. A log row is written.
Read	GET <code>/support/{id}</code>	One ticket. A customer who does not own it is sent back to the list.
Update	POST <code>/support/{id}/respond</code>	Staff save a reply and a status: OPEN, IN_PROGRESS, RESOLVED, or CLOSED.
Delete	POST <code>/support/{id}/delete</code>	The owner or staff can delete that ticket.
Read	GET <code>/support/notifications</code>	The alert log. ADMIN and OPERATIONS_MANAGER.
Update	POST <code>/support/notifications/retry</code>	Retry rows that failed.
Create	POST <code>/forgot-password</code>	If the user exists, save a token and write it into the notification log.
Update	POST <code>/reset-password</code>	If the token is unused and not expired, replace the password.
Read	GET <code>/admin</code>	ADMIN only. Shows current branches and the two create forms.
Create	POST <code>/admin/branches</code>	Add a depot. The row goes in <code>branches</code> .
Create	POST <code>/admin/users</code>	Add a user and a role. Public register cannot create staff.

Screens

- `support/list.jsp` — ticket cards and a button to open a new one.
- `support/form.jsp` — subject and message.
- `support/detail.jsp` — the customer message, and the staff reply form when the viewer is staff.
- `support/notifications.jsp` — channel, recipient, subject, delivery status, and the retry button.
- `auth/forgot-password.jsp` and `auth/reset-password.jsp` — the recovery forms. They use the same photo layout as login.
- `header.jsp` — his menu pass. Customer links and staff links are separate `sec:authorize` blocks.
- `support/admin.jsp` — two cards: add branch, and add user with a role list.
- `auth/login.jsp` and `auth/register.jsp` — sign in and public customer registration. Login posts to Spring Security at `/login`.

Java, in plain words

- `SupportTicket` stores the customer, subject, message, staff reply, status, and timestamps. There is no separate support service class. `SupportController` talks to `SupportTicketRepository` directly, and calls `NotificationService` after a create or a reply.
- `NotificationService.send` writes a `NotificationLog` row. `retryFailed` walks the failed rows and marks another attempt.
- `PasswordResetController` looks up the user by username or email, saves a `PasswordResetToken`, and puts the link text in the log. The reset method checks the token, encodes the new password, and marks the token used.
- `AdminController` lists branches and users, then `addBranch` saves a `Branch` and `addUser` saves a `UserAccount` with one role. Both posts are ADMIN only.
- Booking and payment code also call `NotificationService` when a trip is approved, paid, or updated. The log is the shared record that those events happened.

How to talk about it in the viva

- Say which URL you would open, which JSP draws it, which controller method receives the POST, and which service method applies the rule.
- IT25102266: vehicle management only. Search hides retired cars and cars already booked on those dates. Transfer writes a history row. Delete removes the child rows first. Do not present branches, staff users, or login as this member's use case.
- Maintenance: open job means the car cannot be booked. Close job means it can.
- Booking: pending, then approved or denied. Overlap is checked in the repository.
- Payment: one row per movement. The gateway keeps the last four digits, not the card.
- Dashboard: counts come from the same tables the other members write. Reports do not store their own totals.
- Support: a ticket is saved even when email is only a log row. A failed alert stays visible so it can be retried.

The design is MVC. The controller is the front door, the service is the rules, and the repository is the database. Status values are enums with if-checks. They are not a separate state-machine class.